

Data Warehousing and Business Intelligence

IT3021



Assignment 01

IT23690202

Y3.S1.WD.DS.01.01

Table of Content

1. Task 01: Data set selection	3
1.1 About Data Set	3
1.2 ER diagram	3
1.3 Attribute Description	4
2. Task 02: Preparation of Data Sources	5
2.1 Source 1: orders_source.csv	5
2.2 Source 2: customer_source.xlsx	5
2.3 Source 3: product_source.xlsx	6
2.4 Relationship Between Sources	6
3. Task 03: Solution Architecture	7
3.1 Source Layer	8
3.2 Staging Layer – SSIS ETL package	8
3.3 Integration Layer -SQL view	8
3.4 Data Warehouse Layer – Dimension Table Loading	9
3.5 Data Warehouse Layer – Fact Table Loading	9
3.6 BI & Reporting	9
4. Task 04: Data warehouse design & Development	10
4.1 Fact Table – FactSales	10
4.2 Dimension Descriptions	11
4.3 Assumptions	12
5. Task 05: ETL Development	13
5.1 Package 1: Load_Tables.dtsx	13
5.2 Load Staging Table.....	14
5.3 Load Dimension Table	15
5.3.1 DimCustomer – SCD Type 02	17
5.4 Load Fact Table	18
6. Task 06: ETL Development – Accumulating Fact Tables	20
6.1 Extending FactSales	20
6.2 Creating SalesCompletion table and populating with data	20
6.3 Updating FactSales	21

Task 01: Dataset Selection

1.1 About the Data set

The dataset selected for this assignment is the **Superstore Sales Dataset** obtained from Kaggle. The dataset contains transactional records of product sales in a retail superstore. Each row represents a single order transaction containing customer details, product information, shipping information and financial metrics such as sales and profit.

The dataset includes both categorical and numerical attributes and contains multiple years of data, making it suitable for building a dimensional data warehouse model.

1.2 ER Diagram

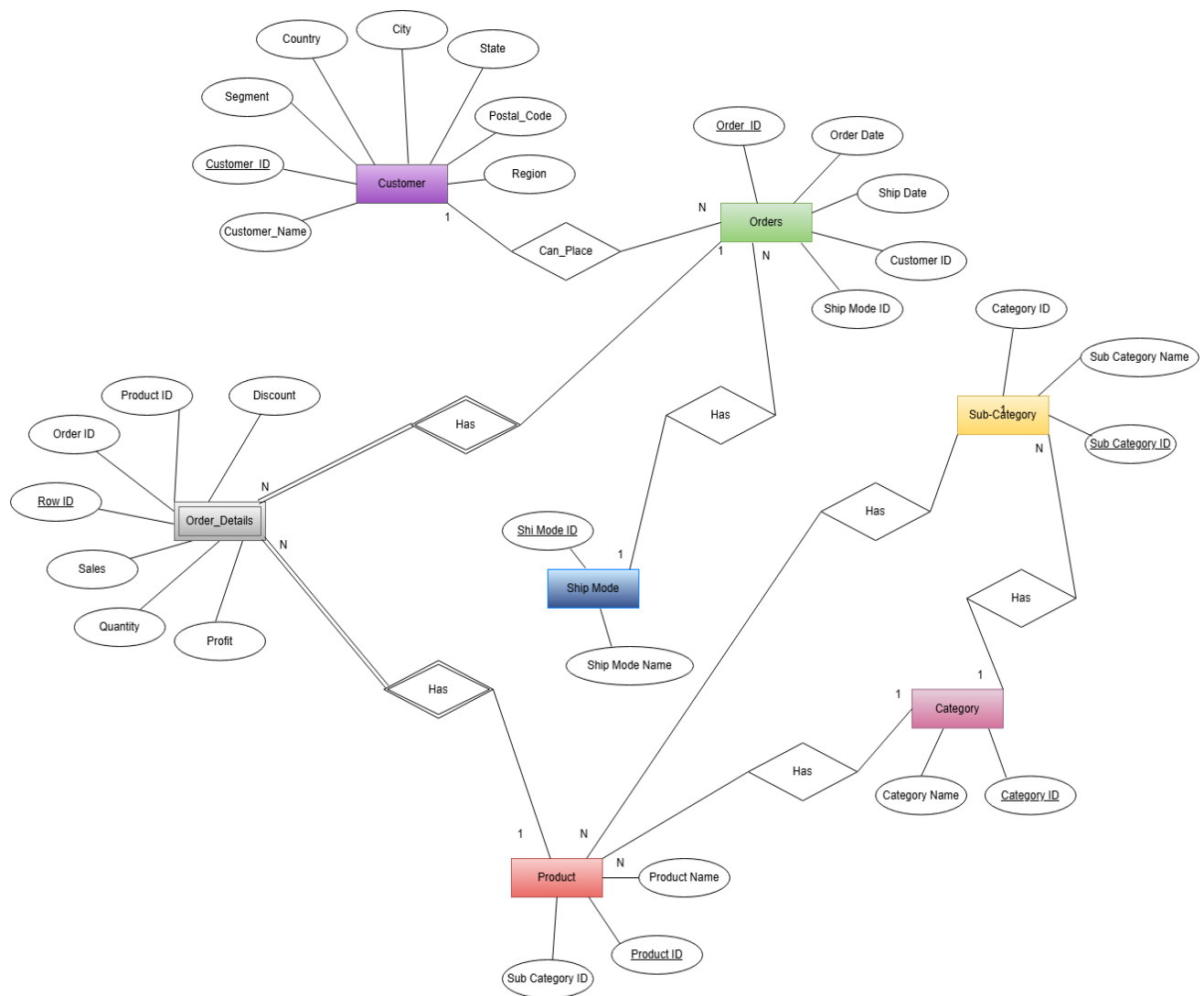


Figure 1. OLTP ER Diagram

1.3 Attribute Description

The descriptions of the attributes as displayed in the Kaggle dataset page.

Attribute	Description
Row ID	Unique ID for each row
Order ID	Unique Order ID for each Customer
Order Date	Order Date for Product
Ship Date	Shipping Date of the Product
Ship Mode	Shipping Mode specified by the customer
Customer ID	Unique ID to identify each customer
Customer Name	Name of the Customer
Segment	The segment where the Customer belongs.
Country	Country of residence of the Customer.
City	City of residence of the Customer.
State	State of residence of the Customer.
Postal Code	Postal Code of every Customer
Region	Region where the Customer belong
Product ID	Unique ID of the Product.
Category	Category of the product ordered.
Sub-Category	Sub-Category of the product ordered.
Product Name	Name of the Product
Sales	Sales of the Product
Quantity	Quantity of the product
Discount	Discount that provided
Profit	Profit or loss incurred

Task 02: Preparation of Data Sources

The original Superstore dataset was separated into multiple source files to simulate real-world operational systems. All are derived from the original data set from Kaggle. This approach allows the ETL process to extract data from multiple heterogeneous sources.

The data sources are logically connected through business keys that represent the operational relationships. The orders_source.csv file contains transactional sales information and includes CustomerID and ProductID fields. The customer_source.xlsx file contains descriptive customer information linked through CustomerID, while the product_source.xlsx file contains product attributes linked through ProductID. During the ETL process, these keys are used to integrate the data sources and populate the corresponding dimension tables in the data warehouse.

Three different data sources were created from the original dataset.

2.1 Source 1: orders_source.csv

property	details
Format	CSV (Comma-Separated Values)
Simulate	Order Transaction System
Stage Table	Stg_Orders
Identifier	RowID (surrogate Key)

This source contains transactional order information including order ID, order date, shipping details and sales measures.

2.2 Source 2: customer_source.xlsx

property	details
Format	Excel Workbook (xlsx)
Simulate	Customer Management System
Stage Table	Stg_Customer
Identifier	RowID (surrogate Key)

This file contains customer attributes such as customer ID, name, segment and geographical information.

2.3 Source 3: product_source.xlsx

property	details
Format	Excel Workbook (xlsx)
Simulate	Product management system
Stage Table	Stg_Product
Identifier	RowID (surrogate Key)

This file contains product attributes including product ID, product name, category and subcategory.

2.4 Relationship Between Sources

Creating Staging table by splinting the original superstore dataset into multiple source files based on subject areas such as orders, customers, and product. As the original dataset was a single flat file, there are no foreign key constraints between tables in this staging layer.

Those files do not share an explicit foreign key column. The sources are linked using the RowID column derived from the original dataset, which act as a passionate surrogate key.

Therefore, the staging tables can be integrated using the **RowID** attribute before loading the data into the dimensional warehouse structure.

The relationship between the staging tables is defined as follows:

Stg_Orders.RowID = Stg_Customer.RowID

Stg_Orders.RowID = Stg_Product.RowID

This enables the ETL process to integrate the data before loading it into the data warehouse.

Task 03: Solution Architecture

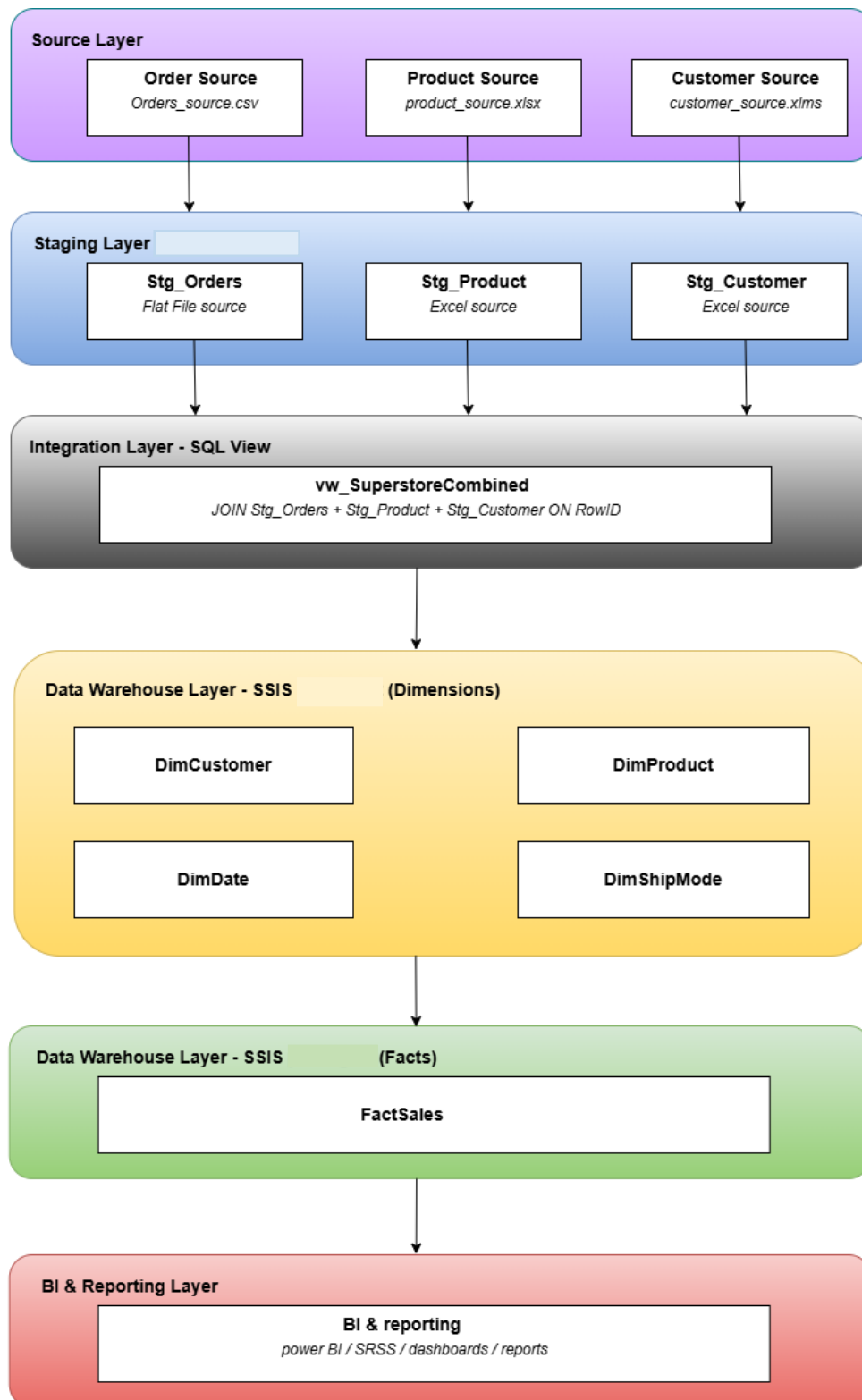


Figure 2. Solution Architectural Diagram

3.1 Source Layer

These source files derived from the original Superstore dataset. These files simulate different operational systems in a real business environment. The source files used are:

- ❖ **orders_source.csv** – contains transactional order data including OrderID, OrderDate, ShipDate, Sales, Quantity, Discount and Profit.
- ❖ **customer_source.xlsx** – contains customer information such as CustomerID, CustomerName, Segment, Country, State, City and Region.
- ❖ **product_source.xlsx** – contains product related information including ProductID, Category, SubCategory and ProductName.

3.2 Staging Layer – SSIS ETL Package

The first SSIS package is responsible for extracting data from the source files and loading them into SQL Server staging tables. Three staging tables are used in this solution:

- ❖ Stg_Orders
- ❖ Stg_Customer
- ❖ Stg_Product

The staging layer temporarily stores the raw imported data before it is transformed and integrated in to the data warehouse. This layer is important because it separates the source data from the dimensional warehouse and allows the ETL process to perform validations, conversions, and cleansing steps without directly affecting the warehouse tables.

3.3 Integration Layer – SQL View

A SQL view **vw_SuperstoreCombined** is created by joining Stg_Orders, Stg_Product and Stg_Customer on RowID. The integrated view acts as a single consolidated data source for loading the dimension and fact table. It simplifies SSIS data flow source queries and ensures consistency as all downstream loads read from the same integrated dataset.

Benefits of the integration view include:

- ❖ simplified ETL queries
- ❖ improved maintainability
- ❖ consistent data access for dimension and fact loading

3.4 Data Warehousing Layer –Dimensions Table Loading

The second SSIS package loads the dimension tables of the data warehouse from the integrated view vw_SuperstoreCombined. The dimensions created in this solution include:

- ❖ DimDate
- ❖ DimShipMode
- ❖ DimProduct
- ❖ DimCustomer

Each dimension stores descriptive attributes that support filtering, grouping, and drill-down analysis in reporting and analytics tools. The data warehouse follows a star schema design where dimensions table are surrounded the central fact table.

3.5 Data Warehouse Layer – Fact Table Loading

The third SSIS package loads the central fact table named **FactSales**. This fact table stores the numerical business measures. It also stores foreign keys that reference the corresponding dimensions such as customer, product, date, and ship mode. This package is executed only after all dimension tables are successfully populated.

3.6 BI & Reporting Layer

The final layer of the architecture is the BI and reporting layer. In this layer, the warehouse data is consumed through tools such as Power BI or SQL Server Reporting Services (SSRS). This layer enables end users to create dashboards, reports, and visual analyses based on clean, integrated, and structured warehouse data.

The BI layer enables better decision making by providing summarized and structured business information.

Task 04: Data warehouse design & Development

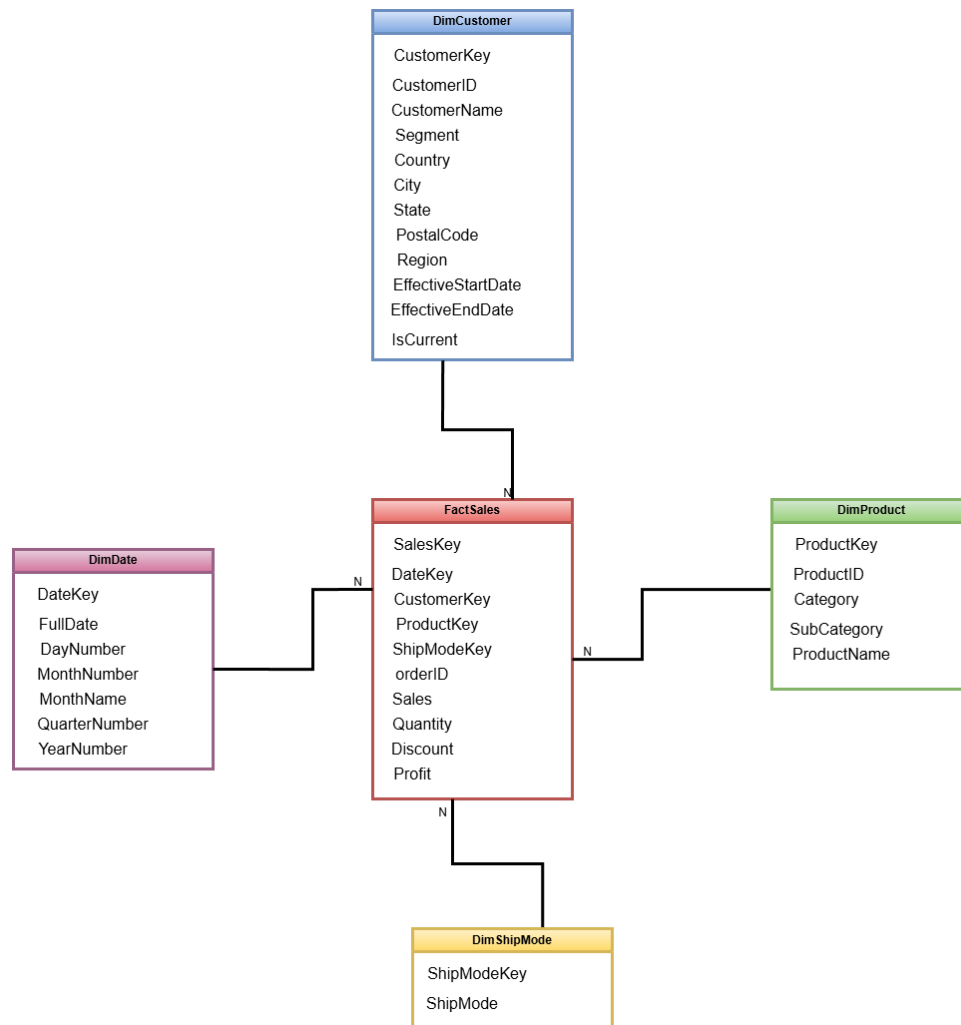


Figure 3. Star Schema of the Superstore Data Warehouse

The dimensional model follows a star schema where FactSales is the central fact table and four-dimension table is connected to it through surrogate keys as DimDate, DimCustomer, DimProduct and DimShipMode. The design follows Kimball's dimensional modelling methodology, optimizing for analytical query performance through denormalization and surrogate key lookups.

4.1 Fact Table: FactSales

The FactSales table Stores the quantitative measures related to each transaction done in the superstore dataset. Each row in this table represents a single sales record corresponding to an order line in the original dataset.

The fact table also contains foreign keys referencing the dimension tables, which allow the fact data to be analyzed from multiple perspectives.
following foreign keys are used in factSales table,

DateKey – Reference DimDate

CustomerKey – Reference DimCustomer

ProductKey – Reference DimProduct

ShipModeKey – Reference DimShipMode

During the ETL process, these keys are resolved from the natural keys present in the staging tables using lookup transformations.

4.2 Dimension Description

- ❖ **DimDate** - represents the time dimension of the warehouse. It contains detailed attributes related to dates derived from the OrderDate field in the dataset. The table includes attributes such as Year, Month, Quarter, Day which enable time-based analysis such as identifying sales trends across months, quarters, or years.
- ❖ **DimCustomer**- contains descriptive information about customers. It includes attributes such as CustomerID, CustomerName, Segment, Country, City, State, PostalCode, and Region. This selected as the Slowly Changing Dimension Type 2, because customer profile and geographical attributes may change over time and such changes should be preserved historically. It keeps track of changes in customer information over time by maintaining historical records. The attributes EffectiveStartDate, EffectiveEndDate, and IsCurrent are used to manage versioning.
- ❖ **DimProduct**- stores product-related descriptive attributes including ProductID, Category, SubCategory, and ProductName. This dimension enables analysis of sales performance by product category, sub-category, or individual products. It allows business users to identify best-selling products, evaluate product category performance, and analyze profitability by product groups.

- ❖ **DimShipMode** - contains the different shipping methods used for order delivery. This dimension includes attributes related to shipping modes such as Standard Class, Second Class, First Class, and Same Day.

4.3 Assumptions

- ❖ The grain of the fact table is one row per sales transaction record from the original Superstore dataset.

The grain of the FactSales table is defined as one record per order line transaction. Each row represents a single product sold in an order. The fact table stores the measurable sales metrics such as Sales, Quantity, Discount, and Profit, while referencing the related dimension keys including DateKey, CustomerKey, ProductKey, and ShipModeKey.

Task 05: ETL Development

5.1 Package 1: Load_Tables.dtsx

Control Flow Task

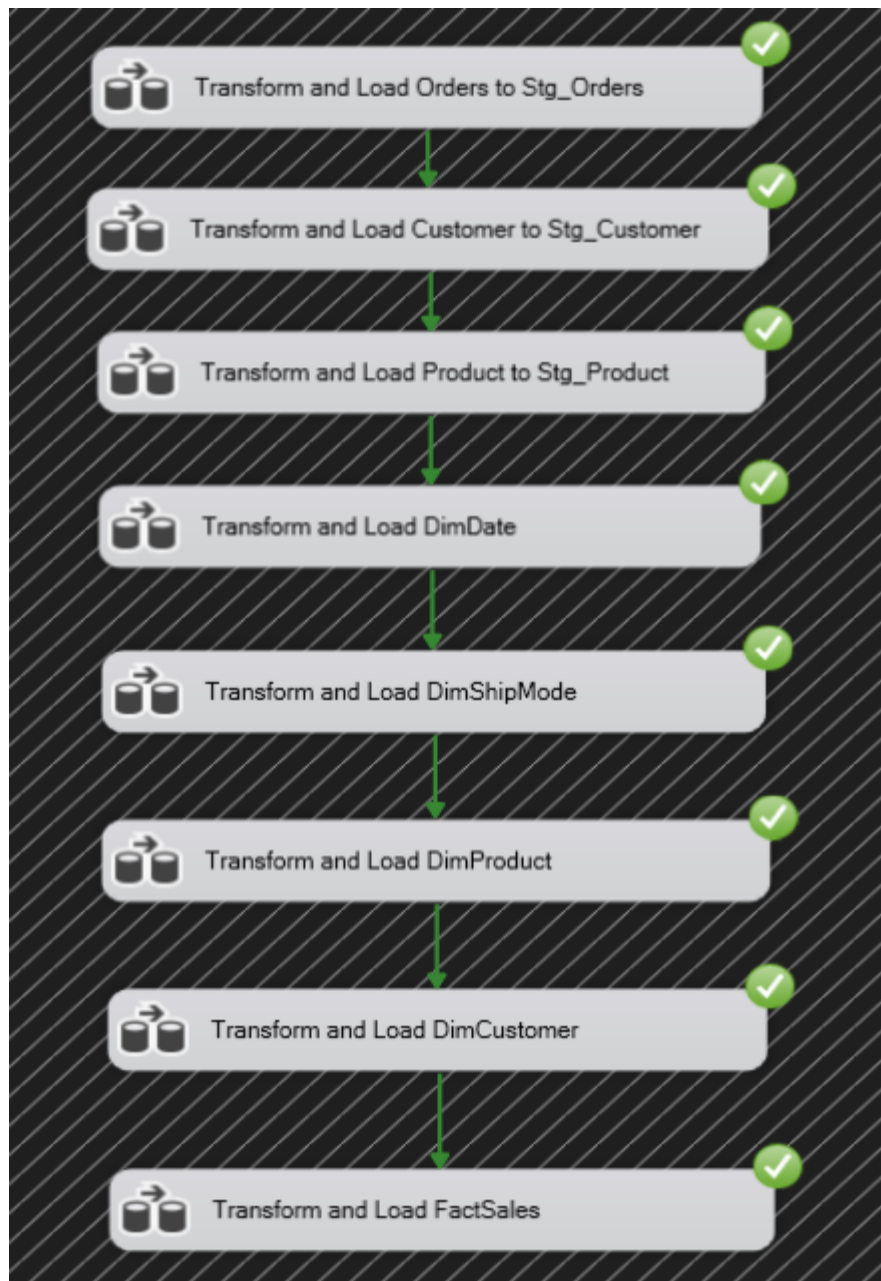


Figure 4. Control Flow Task

The Control Flow in SSIS manages the sequence of ETL operations. Each task in the Control Flow represents a Data Flow Task responsible for extracting, transforming, and loading data into the appropriate tables. The tasks are executed sequentially to maintain referential integrity between tables. This execution order ensures that dimension tables are populated before loading the fact table, allowing foreign key relationships to be maintained.

5.2 Control Flow task 1 - Load Staging Tables

The ETL process for the Superstore Data Warehouse was implemented using SQL Server Integration Services (SSIS). The ETL pipeline extracts transactional data from the Superstore dataset, performs necessary transformations, and loads the processed data into staging tables, dimension tables, and the fact table.

The ETL process follows a structured workflow where data is first loaded into staging tables, followed by the loading of dimension tables, and finally the fact table.

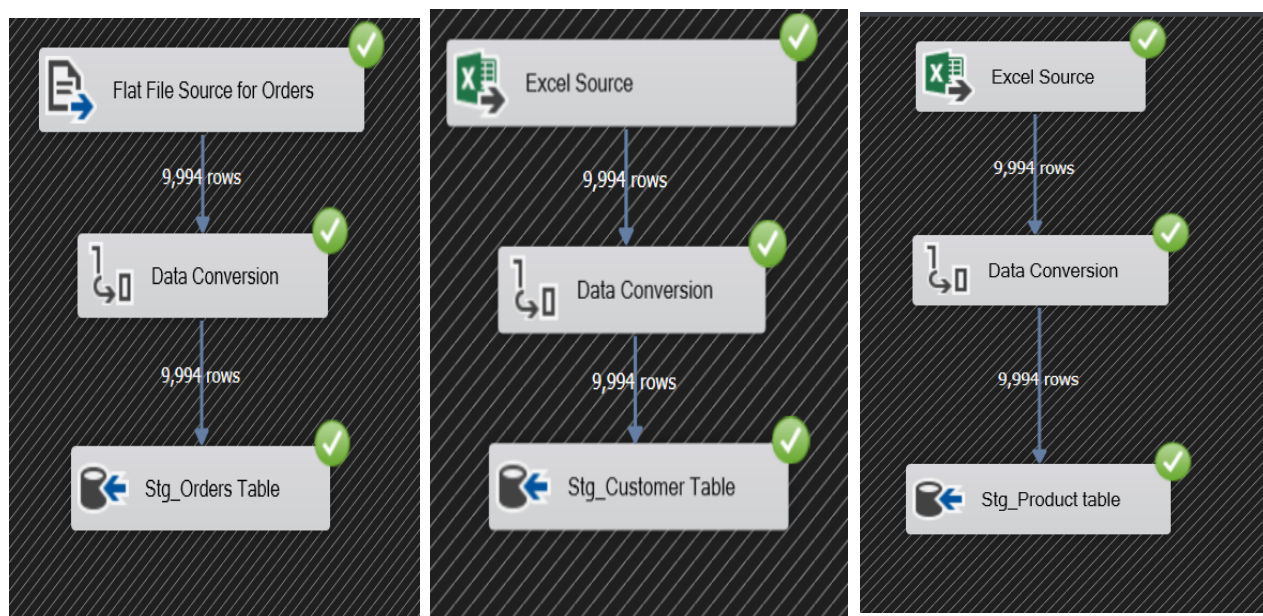


Figure 5. Load_StagingTables SSIS package Data Flow Task

Here I Used 3 source files as Orders_source.csv, custome_source.xlsx and product_source.xlsx. This step stores the raw source data in SQL server staging table before applying warehouse transformation. After loading the staging part the data in integrated using SQL view named vw_SuperstoreCombined.

Flat file source – Read order data from the csv file

Excel Source – Read customer and product data from excel files

Data conversion – Converts data types where required such as converting string values into Unicode numeric format

OLE DB destination – Load the transformed data into staging table

5.3 Control for task 2- Load Dimension Tables

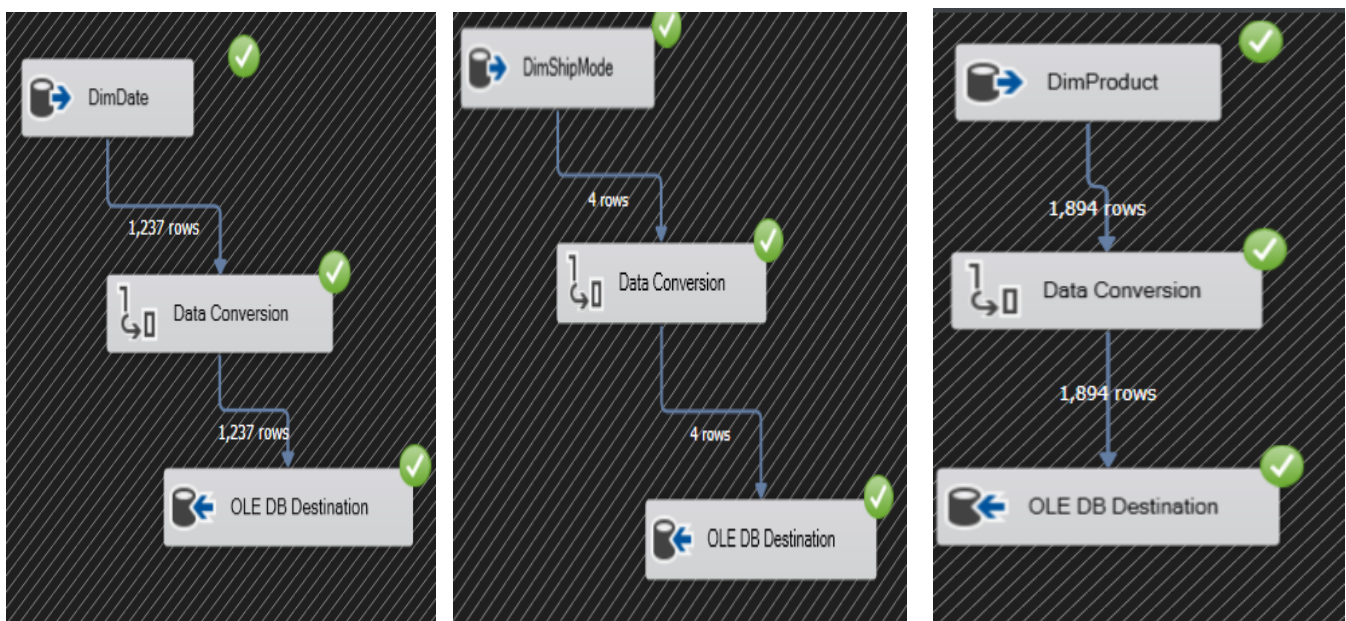


Figure 6. Load_Dimension Tables SSIS package Data Flow Task

The dimension tables in the Superstore Data Warehouse, including DimDate, DimProduct, DimShipMode, and DimCustomer, were populated using SSIS Data Flow transformations. The ETL process begins with extracting dimension-related attributes from the integrated source dataset created from the staging tables. The extracted data is then passed through transformation components to ensure data type compatibility and to prepare the data according to the dimensional schema.

For standard dimension tables such as DimDate, DimProduct, and DimShipMode, the ETL pipeline performs necessary transformations such as data type conversion and column derivation before inserting the records into the respective dimension tables using an OLE DB Destination.

```

--Populate DimDate--
INSERT INTO DimDate
(DateKey, FullDate, DayNumber, MonthNumber, MonthName, QuarterNumber, YearNumber)

SELECT DISTINCT
CAST(FORMAT(CONVERT(date, OrderDate, 101), 'yyyyMMdd') AS INT),
CONVERT(date, OrderDate, 101),
DAY(CONVERT(date, OrderDate, 101)),
MONTH(CONVERT(date, OrderDate, 101)),
DATENAME(MONTH, CONVERT(date, OrderDate, 101)),
DATEPART(QUARTER, CONVERT(date, OrderDate, 101)),
YEAR(CONVERT(date, OrderDate, 101))
FROM Stg_Orders
WHERE OrderDate IS NOT NULL;

```

Figure 7. Load DimDate SQL Task

DimDate is populated using a SQL script executed in SSMS rather than through an SSIS data flow task. The purpose of this script is to extract unique order dates from the staging table and transform them into a structured date dimension suitable for analytical queries.

The SQL script reads the OrderDate field from the Stg_Orders staging table and converts it into a proper DATE format. Since the original data may be stored as text, this conversion ensures that the values are treated as valid SQL date values.

This conversion and date reconstruction logic is more naturally expressed in a single T-SQL statement than through multiple SSIS transformation components.

5.3.1 DimCustomer Data Flow – SCD Type 2

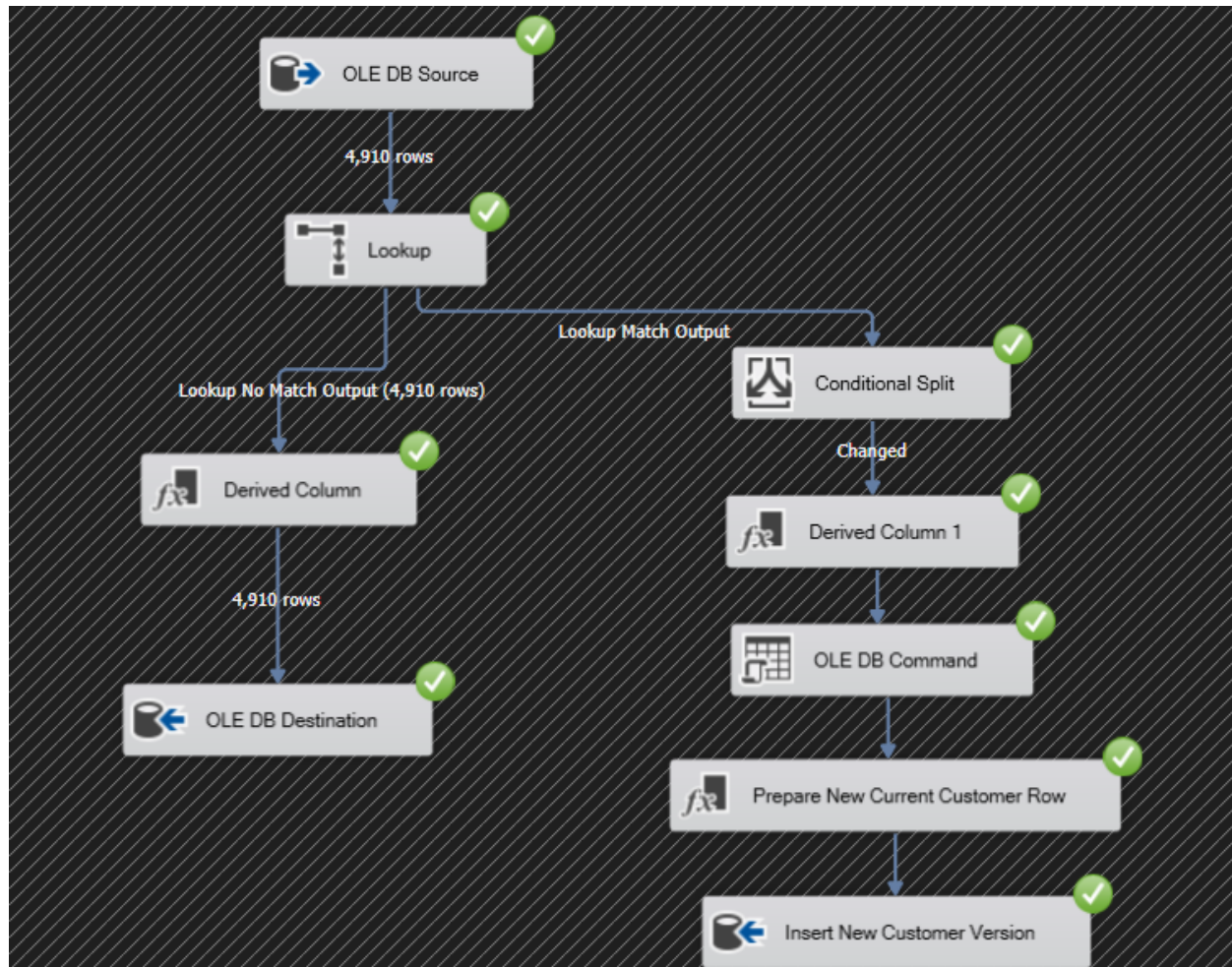


Figure 8. Load DimCustomer SCD Type 2 Data Flow Task

The DimCustomer table was implemented using Slowly Changing Dimension Type 2 to maintain historical customer records whenever customer attributes change. Lookup component compares incoming customer records with existing records in DimCustomer.

The Lookup produces two outputs:

- **No Match Output** – represents new customers that do not exist in the dimension table.
- **Match Output** – represents customers that already exist in the dimension table.

Records from the Lookup No Match Output represent new customers. The records are then inserted into the DimCustomer table using an OLE DB Destination.

For existing customers, records from the Lookup Match Output are sent to a Conditional Split transformation. The Conditional Split checks whether any customer attributes have changed. When change detected, the previous customer record must be expired. An OLE DB command transformation is used to update the existing record. This allows the data warehouse to maintain the full history of customer attribute changes.

This approach ensures that historical customer data is preserved while allowing updated customer records to be stored, which is a key requirement in dimensional data warehouse modeling.

5.4 Control for task 3- Load Fact Table

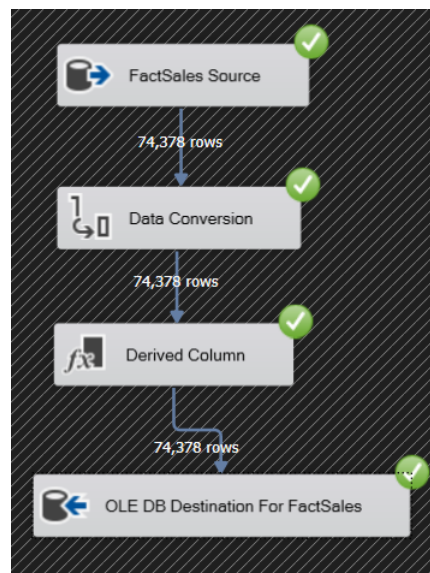


Figure 9. Load Facts Tables SSIS package Data Flow Task

The final step in the ETL pipeline loads transactional data into the FactSales fact table.

The fact table stores measurable business metrics and references dimension tables through foreign keys.

FactSales source – Reads fact data from the integrated source

Data conversion - used to ensure that data types from the source are compatible with the destination table schema.

Derived column – To create or modify columns before loading them into the facts table. In this implementation, the Derived Column transformation generates additional calculated fields required for the fact table.

OLE DB Destination -Loads the transformed records into the FactSales table. In this implementation, the Derived Column transformation generates additional calculated fields required for the fact table.

The SSIS ETL implementation successfully extracts data from the Superstore dataset, transforms it into a consistent format, and loads it into a structured data warehouse schema.

The use of staging tables, dimension tables, and a fact table enables efficient analytical processing and supports business intelligence reporting.

Task 06: ETL Development – Accumulating Fact Tables

6.1 Extending FactSales

```
-- Alter FactSales table to accumulating

ALTER TABLE FactSales
ADD accm_txn_create_time DATETIME NULL,
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours FLOAT NULL;
```

Figure 10. SQL query to extend the FactSales

6.2 Creating SalesCompletion table and populating with data

```
--Task 06 - Create SQL table containing Sales Completion table

CREATE TABLE SalesCompletion(
txn_id INT PRIMARY KEY,
accm_txn_complete_time DATETIME NOT NULL
);

--adding hypothetical data

INSERT INTO SalesCompletion(txn_id, accm_txn_complete_time)
SELECT
    SalesKey,
    DATEADD(
        HOUR,
        ABS(CHECKSUM(NEWID())) % 96 + 48, --random 2-6 period
        accm_txn_create_time
    )
FROM FactSales
WHERE accm_txn_create_time IS NOT NULL;
```

Figure 11. SQL queries for creation and population of SalesCompletion table

6.3 Updating FactSales table



Figure 12. SSIS SQL Task to update the FactSales table

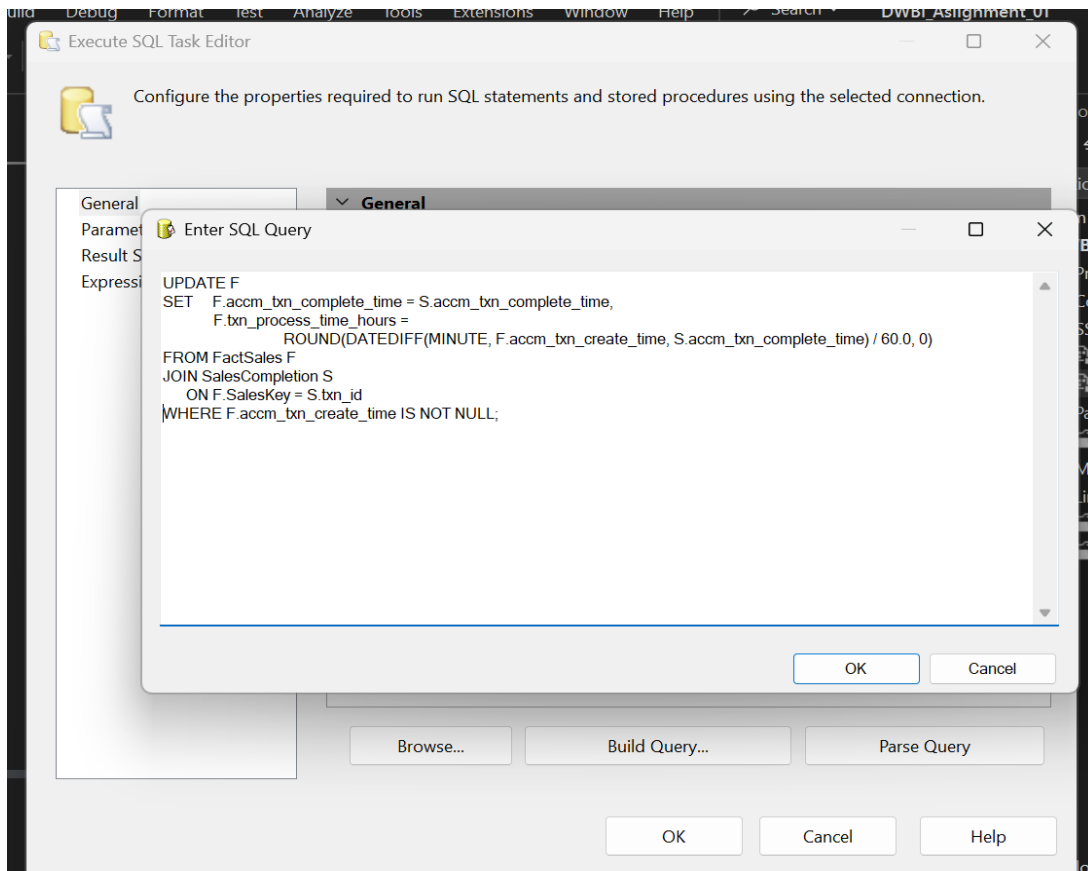


Figure 13. Query used in the SSIS SQL Task to update the FactSales

SalesKey	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	2026-03-25 16:39:34.930	2026-03-29 15:33:58.747	95
2	2026-03-25 16:39:34.930	2026-03-30 03:33:58.747	107
3	2026-03-25 16:39:34.930	2026-03-29 10:33:58.747	90
4	2026-03-25 16:39:34.930	2026-03-30 02:33:58.747	106
5	2026-03-25 16:39:34.930	2026-03-30 12:33:58.747	116
6	2026-03-25 16:39:34.930	2026-03-29 03:33:58.747	83
7	2026-03-25 16:39:34.930	2026-03-28 18:33:58.747	74
8	2026-03-25 16:39:34.930	2026-03-30 00:33:58.747	104
9	2026-03-25 16:39:34.930	2026-03-28 20:33:58.747	76
10	2026-03-25 16:39:34.930	2026-03-27 14:33:58.747	46
11	2026-03-25 16:39:34.930	2026-03-28 16:33:58.747	72
12	2026-03-25 16:39:34.930	2026-03-28 18:33:58.747	74
13	2026-03-25 16:39:34.930	2026-03-30 20:33:58.747	124
14	2026-03-25 16:39:34.930	2026-03-30 18:33:58.747	122
15	2026-03-25 16:39:34.930	2026-03-31 06:33:58.747	134
16	2026-03-25 16:39:34.930	2026-03-30 10:33:58.747	114
17	2026-03-25 16:39:34.930	2026-03-28 05:33:58.747	61
18	2026-03-25 16:39:34.930	2026-03-30 11:33:58.747	115
19	2026-03-25 16:39:34.930	2026-03-27 21:33:58.747	53
20	2026-03-25 16:39:34.930	2026-03-30 23:33:58.747	127

Figure 14. Sample Rows of the updated FactSales table